

Concurrency Control

Mutual exclusion and synchronisation

Threads and Objects

- Objects
 - don't belong to threads
 - can be accessed simultaneously by multiple threads
 - must be designed to operate correctly when called by multiple threads (thread safe)
- Ensuring “thread safety” requires
 - preventing corruption of object state by simultaneous operations
 - control of interleaving of thread operations on the object
 - use of concurrency control constructs

Concurrency Control

- Two key aspects to concurrency control
 - Mutual exclusion
 - Preventing object / resource from being accessed by more than one thread at once
 - Needed to prevent corruption of state in presence of multiple client updates
 - Implemented through locking
 - Synchronization
 - Imposes order on interleaving of operations of multiple threads (usually for correctness)
 - Implemented using semaphores, condition variables or monitors

Common Concurrency Problems

- Deadlock

- A situation where two or more processes are unable to proceed because each is waiting for one of the others to do something
- Avoid by
 - always acquiring / releasing locks in the same order
 - ensuring that all Wait()'s have a matching Signal()

- Livelock

- When two or more processes continuously change their state in response to changes in the other process(es) without doing any useful work.
- Similar to deadlock in that no progress is made but differs in that neither process is blocked or waiting for anything.

Common Concurrency Problems

- Race Conditions

- Anomalous behaviour due to unexpected critical dependence on the relative timing of events.
- For example, if one process writes to a file while another is reading from the same location then the data read may be the old contents, the new contents or some mixture of the two depending on the relative timing of the read and write operations.
- Can occur if scope of locks is too small and some code left outside critical sections

Mutual Exclusion (Python)

Locks

Re entrant locks

Locks

- The implementation of mutual exclusion
- Use to limit access to a section of code, known as a critical section.

Basic threading – errors

```
1.  import threading
2.  from time import sleep

3.  def threadMain(i=2):
4.      sleep(0.5)
5.      print ('thread',i,'sleeping for',i,'secs')

6.  def main():
7.      threads = []
8.      threadIds = range(4)

9.      for i in threadIds:
10.         t = threading.Thread(target = threadMain, args = [i])
11.         threads.append(t)
12.
13.     for t in threads:
14.         t.start()

15.     for t in threads:
16.         t.join()

17. if __name__ == '__main__':
18.     main()
```

threadthreadthreadthread
1230 sleeping forsleeping
forsleeping forsleeping for
1230 secssecssecssecs

Locking

- Objects or methods can have an associated lock
- The 1st thread to execute the lock statement will be granted the lock and can execute the protected section of code
- Whilst the 1st thread has the lock, any other threads executing the lock statement will block until the lock is released.
- Waiting threads will compete for the lock once the lock has been released

Thread safe printing

```
1. printLock = threading.Lock() #lock to be used by all threads

2. def safePrint(*args):
3.     global printLock
4.     printLock.acquire()
5.     print(*args)
6.     printLock.release()
```

```
Starting main programme
thread 0 sleeping for 0.5 secs
thread 1 sleeping for 0.5 secs
thread 2 sleeping for 0.5 secs
thread 3 sleeping for 0.5 secs
Ending main programme
```

Thread safe printing

```
1. printLock = threading.Lock() #lock to be used by all threads

2. def safePrint(*args):
3.     global printLock
4.     printLock.acquire()
5.     print(*args)
6.     printLock.release()
```

```
Starting main programme
thread 0 sleeping for 0.5 secs
thread 2 sleeping for 0.5 secs
thread 1 sleeping for 0.5 secs
thread 3 sleeping for 0.5 secs
Ending main programme
```

Locking, common pitfalls

- Consider the case where the task fails, within the lock.

```
1. def safePrint(*args):  
2.     global printLock  
3.     printLock.acquire()  
4.     print(*args)  
5.     printLock.release()
```

Locking, common pitfalls

- Always code for possible failures

```
1. def safePrint(*args):  
2.     global printLock  
3.     printLock.acquire()  
4.     try:  
5.         print(*args)  
6.     finally:  
7.         printLock.release()
```

Locking, common pitfalls

- Always code for possible failures

```
1. def safePrint(*args):  
2.     global printLock  
3.     with printLock:  
4.         print(*args)
```

Unsynchronized Shared Memory

```
1. class SharedObject:
2.     def getData(self):
3.         print('---','---')

4.     def setData(self):
5.         print('***','***')

6. sharedObject = SharedObject()

7. def consume():
8.     global sharedObject
9.     for i in range(4):
10.         sharedObject.getData()

11. def produce():
12.     global sharedObject
13.     for i in range(4):
14.         sharedObject.setData()
```

Unsynchronized Shared Memory

Main

```
1. def main():
2.     print('Starting main programme')

3.     threads = []
4.     threads.append(threading.Thread(target=produce))
5.     threads.append(threading.Thread(target=consume))
6.
7.     for t in threads:
8.         t.start()

9.     for t in threads:
10.        t.join()

11.    print('Ending main programme')
```

```
Starting main programme
***---  ***---

***---  ***---

***---  ***---

***---  ***---

Ending main programme
```

Unsynchronized Shared Memory, with locks

```
1. class SharedObject:
2.     def __init__(self):
3.         self.lock = threading.Lock()

4.     def getData(self):
5.         self.lock.acquire()
6.         try:
7.             print('---', '---')
8.         finally:
9.             self.lock.release()

10.    def setData(self):
11.        self.lock.acquire()
12.        try:
13.            print('***', '***')
14.        finally:
15.            self.lock.release()
```

Unsynchronized Shared Memory, with locks

```
1. sharedObject = SharedObject()

2. def consume():
3.     global sharedObject
4.     for i in range(4):
5.         sharedObject.getData()

6. def produce():
7.     global sharedObject
8.     for i in range(4):
9.         sharedObject.setData()
```

```
Starting main programme
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
Ending main programme
```

Unsynchronized Shared Memory, with locks

```
Starting main programme
***  ***
---  ---
---  ---
***  ***
***  ***
---  ---
***  ***
---  ---
Ending main programme
```

Limitations with simple locking

- Simple locks have two states, locked and unlocked
- They have no knowledge of which thread holds the lock
- Consider the case where an object is locked during multiple accesses to its data.

Unsynchronized shared memory, with multiple accesses

```
1. class SharedObject:
2.     def __init__(self):
3.         self.lock = threading.Lock()

4.     def getDataA(self):
5.         self.lock.acquire()
6.         try:
7.             print('-A-')
8.         finally:
9.             self.lock.release()

10.    def getDataB(self):
11.        self.lock.acquire()
12.        try:
13.            print('-B-')
14.        finally:
15.            self.lock.release()
```

Unsynchronized shared memory, with multiple accesses

```
1. def setData(self):  
2.     self.lock.acquire()  
3.     try:  
4.         print('***')  
5.     finally:  
6.         self.lock.release()  
  
7. def getAllData(self):  
8.     self.getDataA()  
9.     self.getDataB()
```

Unsynchronized shared memory, with multiple accesses

```
1. sharedObject = SharedObject()

2. def consume():
3.     global sharedObject
4.     for i in range(4):
5.         sharedObject.setData()

6. def produce():
7.     global sharedObject
8.     for i in range(4):
9.         sharedObject.getAllData()
```

```
Starting main programme
***
-A-
***
-B-
***
-A-
***
-B-
-A-
-B-
-A-
-B-
Ending main programme
```

Unsynchronized shared memory, with multiple accesses

```
1. def getAllData(self):  
2.     self.lock.acquire()  
3.     try:  
4.         self.getDataA()  
5.         self.getDataB()  
6.     finally:  
7.         self.lock.release()
```

```
Starting main programme  
***
```

Re-entrant locks

- A re-entrant lock does not block a method on the same thread
 - It permits a thread to re-enter a lock if it already has it acquired.



Unsynchronized shared memory, with re-entrant locks

```
1. class SharedObject:
2.     def __init__(self):
3.         self.lock = threading.RLock()

4.     def getDataA(self):
5.         self.lock.acquire()
6.         try:
7.             print('-A-')
8.         finally:
9.             self.lock.release()

10.    def getDataB(self):
11.        self.lock.acquire()
12.        try:
13.            print('-B-')
14.        finally:
15.            self.lock.release()
```

Unsynchronized shared memory, with re-entrant locks

```
1.     def setData(self):
2.         self.lock.acquire()
3.         try:
4.             print('***')
5.         finally:
6.             self.lock.release()

7.     def getAllData(self):
8.         self.lock.acquire()
9.         try:
10.            self.getDataA()
11.            self.getDataB()
12.        finally:
13.            self.lock.release()
```

```
Starting main programme
***
-A-
-B-
***
-A-
-B-
***
-A-
-B-
***
Ending main programme
```

Unsynchronized shared memory, with re-entrant locks

•

```
Starting main programme
***
-A-
-B-
***
***
-A-
-B-
-A-
-B-
***
-A-
-B-
Ending main programme
```

Locking - Disadvantages

- Locking is essential to preventing conflicting updates but can adversely affect performance
- If scope of lock is too great
 - Forces some threads to wait
 - Decreases potential concurrency and throughput / performance
- If scope of lock is too small
 - Forces reacquisition of locks, leading increased lock contention and reduced throughput / performance
 - Can result in difficult-to-debug race conditions

Advanced Locking

- Mutual exclusion
 - Not needed if all threads are reading values
 - Needed if at least one thread is writing values
- Read Write lock
 - allows concurrent read access for multiple threads, or
 - write access for a single thread
 - uses separate locks for readers and writers; clients must specify which lock they want

Mutual Exclusion (C++ 11)

Locks

Locks

- classes of mutexes
 - mutex
 - recursive_mutex
 - For re-entrant locks
 - timed_mutex
 - For time limited locking
 - recursive_timed_mutex
- Operations
 - lock
 - Blocks the calling thread until the lock is available
 - try_lock
 - Attempts to get ownership of the lock without blocking
 - True = has ownership
 - unlock
 - Releases a lock

Shared Object with locks

```
1. #include <mutex>

2. class SharedObject {
3.     mutex mx;
4. public:
5.     void getData() {
6.         mx.lock();
7.         cout << "---" << endl;
8.         mx.unlock();
9.     }
10.    void setData() {
11.        mx.lock();
12.        cout << "***" << endl;
13.        mx.unlock();
14.    }
15.};
```

Utility templates

- `lock_guard`
 - locks a mutex on object creation, and unlocks on object destruction.
- `unique_lock`
 - creates an object to control mutexes
- `try_lock_for`
 - Times out after time `t`
- `try_lock_until`
 - Time out at a specific time

Shared Object with locks

```
1. #include <mutex>

2. class SharedObject {
3.     mutex mx;
4. public:
5.     void getData() {
6.         lock_guard<mutex> lg(mx);
7.         cout << "---" << endl;
8.     }
9.     void setData() {
10.        lock_guard<mutex > lg(mx);
11.        cout << "****" << endl;
12.    }
13.};
```

Unique locks

- Example of using `unique_lock` as a way to control locks

```
1. void getData() {  
2.     while(true) {  
3.         unique_lock<mutex> ul(mx, try_to_lock_t());  
4.         if (ul)  
5.             cout << "---" << endl;  
6.             break;  
7.         else  
8.             this_thread::sleep_for(chrono::seconds(1));  
9.     }  
10. }
```

Synchronization (Python)

Events

Conditions

Synchronization

- Needed when a thread wishes to perform an operation which can only sensibly(safely) be performed if another thread has itself taken some other action or is in some defined state
- Example : A bounded buffer has a 2 condition synchronization
 - producer thread must not attempt to put data in buffer if it is full
 - consumer thread must not attempt to get data from buffer if it is empty
 - Is mutual exclusion necessary?

Unsyncronised threads

```
1. def physics():
2.     safePrint('Physics - begin')
3.     sleep(random())
4.     safePrint('Physics - end')

5. def renderer():
6.     safePrint('Rendering - begin')
7.     sleep(random())
8.     safePrint('Rendering - end')
```

Unsyncronised threads

```
1. def main():
2.     print('Starting main programme')
3.     threads = []
4.     threads.append(threading.Thread(target=physics))
5.     threads.append(threading.Thread(target=renderer))

6.     for t in threads:
7.         t.start()

8.     for t in threads:
9.         t.join()

10.    print('Ending main programme')
```

Starting main programme
Physics - begin
Rendering - begin
Rendering - end
Physics - end
Ending main programme

Synchronised threads, using a busy loop

```
1. physicsFinished = False

2. def physics():
3.     global physicsFinished
4.     safePrint('Physics - begin')
5.     sleep(random())
6.     safePrint('Physics - end')
7.     physicsFinished = True

8. def renderer():
9.     global physicsFinished
10.    while (NOT physicsFinished):
11.        sleep(0.1)
12.        safePrint('Rendering - begin')
13.        sleep(random())
14.        safePrint('Rendering - end')
15.        physicsFinished = False
```

```
Starting main programme
Physics - begin
Physics - end
Rendering - begin
Rendering - end
Ending main programme
```

Events

- An event is a simple synchronisation object
- The event represents a low level flag
- A thread can wait on an event, set an event, or clear an event.



Synchronised threads, using an event

```
1. physicsFinished = threading.Event()

2. def physics():
3.     global physicsFinished
4.     safePrint('Physics - begin')
5.     sleep(random())
6.     safePrint('Physics - end')
7.     physicsFinished.set()

8. def renderer():
9.     global physicsFinished
10.    physicsFinished.wait()
11.    safePrint('Rendering - begin')
12.    sleep(random())
13.    safePrint('Rendering - end')
14.    physicsFinished.clear()
```

```
Starting main programme
Physics - begin
Physics - end
Rendering - begin
Rendering - end
Ending main programme
```

Unsynchronized shared memory, with locks

```
1. sharedObject = SharedObject()

2. def consume():
3.     global sharedObject
4.     for i in range(4):
5.         sharedObject.getData()

6. def produce():
7.     global sharedObject
8.     for i in range(4):
9.         sharedObject.setData()
```

```
Starting main programme
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
Ending main programme
```

Unsynchronized shared memory, with locks

```
1. sharedObject = SharedObject()

2. def consume():
3.     global sharedObject
4.     for i in range(4):
5.         sleep(random())
6.         sharedObject.getData()

7. def produce():
8.     global sharedObject
9.     for i in range(4):
10.        sleep(random())
11.        sharedObject.setData()
```

```
Starting main programme
--- ---
*** ***
*** ***
--- ---
--- ---
*** ***
--- ---
*** ***
Ending main programme
```

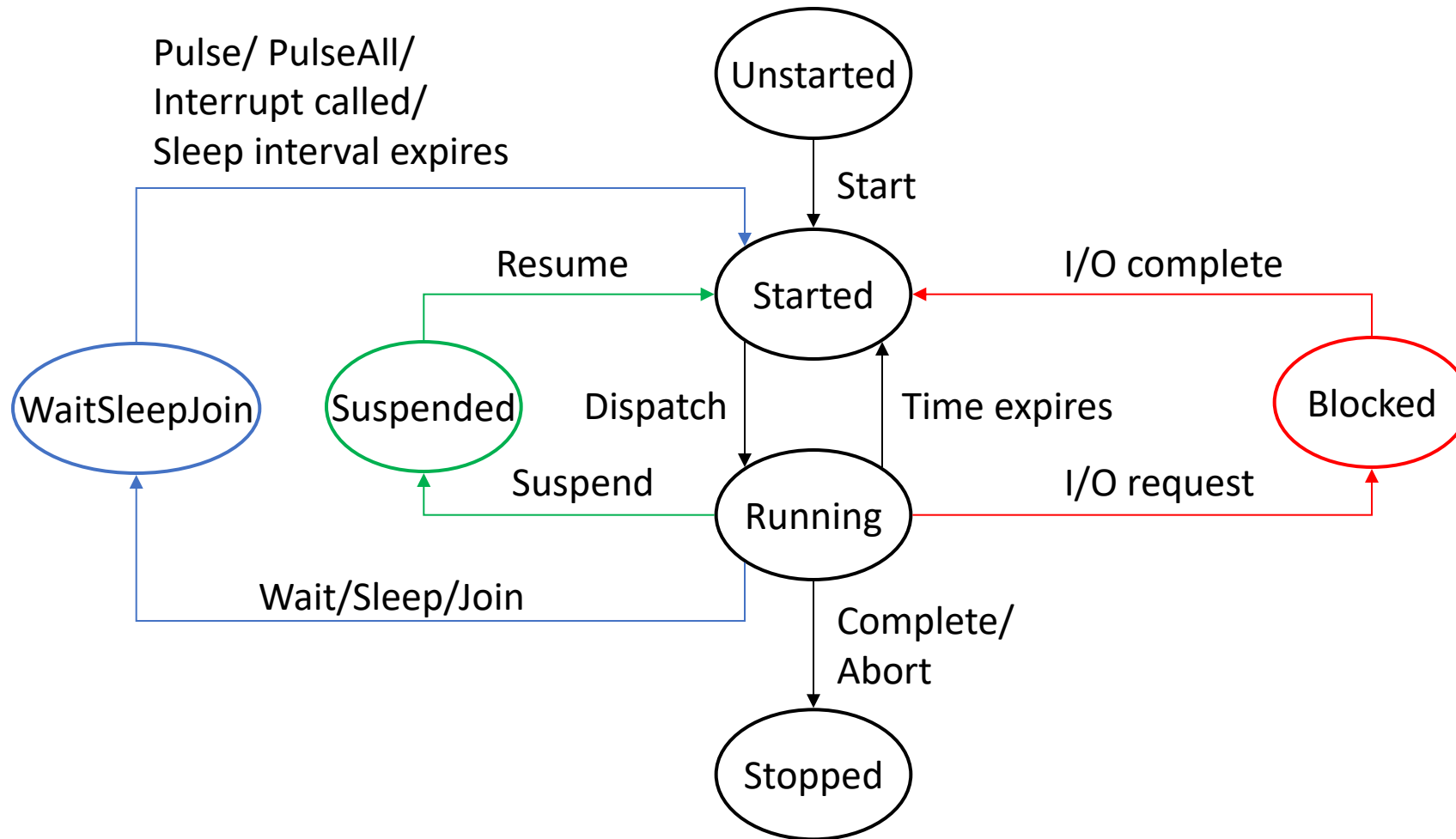
Conditions (monitors)

- Condition
 - Code construct that provides both mutual exclusion and condition synchronization
 - Acts like a 'smart' lock
- Thread enters the condition and attempts to acquire the lock
 - If successful, the thread executes under mutual exclusion; the thread frees lock when finished
 - If unsuccessful, the thread blocks until the condition is available

Conditions

- When inside a condition a thread can
 - wait
 - Thread blocks and releases lock, freeing monitor to other threads
 - When the condition becomes true, the thread unblocks, reacquires the lock and executes under mutual exclusion
 - signal
 - Notifies waiting threads that condition is true
 - Releases lock, freeing condition for other threads

Thread Life cycle



Synchronized Shared Memory

```
1. class SharedObject:
2.     def __init__(self):
3.         self.__empty = True
4.         self.condition = threading.Condition()
5.
6.     def getData(self):
7.         self.condition.acquire()
8.         try:
9.             while True:
10.                 if not self.__empty:
11.                     self.__empty = True
12.                     print('---', '---')
13.                     return
14.                 self.condition.wait()
15.         finally:
16.             self.condition.notify()
17.             self.condition.release()
```

Synchronized Shared Memory

```
1.     def setData(self):
2.         self.condition.acquire()
3.         try:
4.             while True:
5.                 if self.__empty:
6.                     self.__empty = False
7.                     print('***', '***')
8.                     return
9.                 self.condition.wait()
10.        finally:
11.            self.condition.notify()
12.            self.condition.release()
```

Synchronized shared memory

```
1. sharedObject = SharedObject()

2. def consume():
3.     global sharedObject
4.     for i in range(4):
5.         sleep(random())
6.         sharedObject.getData()

7. def produce():
8.     global sharedObject
9.     for i in range(4):
10.        sleep(random())
11.        sharedObject.setData()
```

```
Starting main programme
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
Ending main programme
```

Conditions

- Provides a mechanism that synchronizes access to objects
- Acquire
 - Acquires a lock for an object
 - Marks the beginning of a critical section
 - No other thread can enter the critical section
- Wait
 - Releases the lock on an object, permitting other threads access to the object.
 - The calling thread waits while another thread accesses the object.
- Notify/ NotifyAll
 - Sends a signal to one or more waiting threads
 - The signal tells the waiting thread that the owner is ready to release the lock.
 - The waiting thread is placed in the ready queue
- Release
 - Releases the lock on an object
 - Marks the end of a critical section

Shared Memory

- What other architectural options are there are sharing data?



Synchronization (C++ 11)

Events

Conditions

Condition variables

- Condition variables are a synchronization primitive that block a thread until
 - Receives a notification
 - Timeout expires
 - Receives a wakeup
- Unique lock is a wrapper class for mutexes that allows their use with condition variables
 - `unique_lock <mutex> lock(mx);`

Synchronized Shared Memory

```
1.  class SharedObject {
2.  private:
3.      mutex mx;
4.      condition_variable cv;
5.      bool empty;

6.  public:
7.      SharedObject() : empty(true) {}

8.      void getData() {
9.          unique_lock<mutex> lk(mx);
10.         while (true) {
11.             if (!empty) {
12.                 empty = true;
13.                 cout << "---" << endl;
14.                 break;
15.             }
16.             cv.wait(lk);
17.         }
18.         cv.notify_all();
19.     }
```

Synchronized Shared Memory

```
1. void setData() {  
2.     unique_lock<mutex> lk(mx);  
3.     while (true) {  
4.         if (empty) {  
5.             empty = false;  
6.             cout << "****" << endl;  
7.             break;  
8.         }  
9.         cv.wait(lk);  
10.    }  
11.    cv.notify_all();  
12. }
```

Synchronized shared memory

```
1. SharedObject shareObject;

2. auto consume = [] () {
3.     for (int i=0; i<4; i++)
4.         shareObject.getData();
5.     return 0;
6. };

7. auto produce = [] () {
8.     for (int i=0; i<4; i++)
9.         shareObject.setData();
10.    return 0;
11.};
```

```
Starting main programme
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
***  ***
---  ---
Ending main programme
```

Futures and Promises

- Promises and futures provide a mechanism to safely transfer values between threads.
- A promise is an object that can store a value which can be safely retrieved by a future object, in another thread.
- Promises are particularly useful at retrieving intermediate data from within a thread, and not having to wait until thread completion.
- Note: future objects are an intrinsic part of the async function mechanism, and do not require promises in that context.

Futures and Promises

```
1.  #include <iostream>
2.  #include <future>
3.  using namespace std;

4.  auto produce = [] (promise<int>& p1, promise<int>& p2) {
5.      int item = 1;
6.      this_thread::sleep_for(chrono::milliseconds(50));
7.      cout << "Producing " << item << endl;
8.      p1.set_value(item);

9.      item = 2;
10.     this_thread::sleep_for(chrono::milliseconds(50));
11.     cout << "Producing " << item << endl;
12.     p2.set_value(item);
13.
14.     return 0;
15. };
```

Futures and Promises

```
1. auto consume = [](future<int>& f1, future<int>& f2) {
2.     cout << "Waiting ..." << endl;
3.     int item = f1.get();
4.     cout << "Consumed " << item << endl;

5.     item = f2.get();
6.     cout << "Consumed " << item << endl;
7.     return 0;
8. };

9. void main(int, char**) {
10.    promise<int> p1, p2;
11.    thread t1(produce, ref(p1), ref(p2));
12.    thread t2(consume, p1.get_future(), p2.get_future());
13.    t1.join();
14.    t2.join();
15.}
```